OPEN SOURCE · MIT · VERSION 2.1.0

ng-checkup.

The complete manual

A one-minute modernization report for any Angular codebase. This manual describes everything the package does in version 2.1 (2.1.0): every command and option, every area and how it is graded, every measure the baseline records, every report format, every integration (GitHub, GitLab, Azure DevOps, Bitbucket, VS Code, MCP), the programmatic API, what the tool sends over the network, how it works inside, and how it is released.

Package	`ng-checkup-cli` on npm (the command it installs is `ng-checkup`)
Version	2.1.0, released 6 November 2026
Licence	MIT
Requirements	Node.js 22 or newer; git for remote repositories, history and `--since`
Source	github.com/stefankuburovic/ng-checkup
GitHub Action	`stefankuburovic/ng-checkup@v2`
GitLab component	`gitlab.com/stefankuburovic/ng-checkup/ng-checkup@2`
Editor extension	`stefankuburovic.ng-checkup-vscode` (VS Code Marketplace, Open VSX)
Author	Stefan Kuburovic, stefankuburovic.github.io

Contents

1. What ng-checkup is
2. Installing and running it
3. The command line
4. The configuration file
5. Grades, the score and severities
6. The areas, one by one
7. The baseline and every measure
8. Reports and output formats
9. Setting a project up: `init` and `ng add`
10. Continuous integration
11. Only what a branch changed: `--since`
12. Watch mode, history and the trend
13. The upgrade plan and the migration plans
14. Monorepos and projects
15. Remote repositories and live sites
16. Many repositories: `fleet`
17. The client report
18. The VS Code extension
19. The MCP server for editor agents
20. The programmatic API
21. What leaves your machine
22. How it works inside
23. Quality assurance: tests and the corpus
24. Versions, tags and the release process
25. Troubleshooting and FAQ
26. What changed in 2.1 and 2.0
27. Related packages

1. What ng-checkup is

Point ng-checkup at an Angular repository and within seconds it answers three questions:

- **How far is this code from current Angular?** The version gap, zoneless readiness, standalone components, signals, built-in control flow, the test runner, strictness.
- **What breaks on the next upgrade?** Deprecated APIs as the installed version marks them, imports the next major no longer exports, third-party packages that do not support the next Angular.
- **Where is the work?** Every finding names the files and lines behind it, and a plan lists the steps from the installed Angular to the latest, one major at a time.

It reads the source; it never builds, serves, installs or runs anything from the repository. That is what lets it run on a fresh clone, on a repository that does not compile, in CI before `npm ci`, and on a repository it has never seen.

Who it is for. Teams that want a guard in CI against new technical debt; tech leads planning an upgrade; consultants who need a fast, reproducible health check of a client codebase; agencies and companies with many front ends who need to know which one to look at first.

What it is not. It is not a linter (it does not enforce a style, and it does not report a line it cannot tie to modernization, upgrades or safety), not a test-coverage tool (it never runs tests), and not a bundler (it reads build stats, it does not build).

The areas at a glance

ID	AREA	GRADED
`primo-nde`	Primo NDE migration (only for an Ex Libris Primo VE package)	yes, when present
`angularjs`	AngularJS migration (only with AngularJS code or packages)	yes, when present
`version`	Angular version & tooling	yes
`deprecated-apis`	Deprecated APIs	yes
`dependencies`	Third-party compatibility	yes
`change-detection`	Change detection & zoneless readiness	yes
`components`	Components & templates	yes
`state`	State management	yes
`architecture`	Architecture	yes
`testing`	Testing	yes
`type-safety`	Type safety	yes
`security`	Security	yes
`translations`	Translations	no: listed only
`accessibility`	Accessibility	no: listed only
`circular-deps`	Circular dependencies	yes
`bundle`	Bundle size	yes

Twelve areas are graded on every repository; translations and accessibility report findings and measures without a grade; one legacy area (AngularJS or Primo NDE) comes first when the repository has legacy code.

2. Installing and running it

Nothing has to be installed. In the root of an Angular CLI or Nx workspace:

```
npx ng-checkup-cli
```

`npx` downloads the package once into its cache and runs the `ng-checkup` command. The three commands most people need:

```
# 1. The summary in the terminal
npx ng-checkup-cli

# 2. The full report as one self-contained page
npx ng-checkup-cli --report checkup.html

# 3. Record today's numbers, then fail CI only when one gets worse
npx ng-checkup-cli baseline
npx ng-checkup-cli --fail-on regression
```

Without `node_modules`

The project does not have to be installed. When `node_modules` is missing, exact versions come from the lockfile (`package-lock.json`, `yarn.lock` or `pnpm-lock.yaml`), and the Angular, NgRx and RxJS type definitions the deprecated-API check reads are downloaded once for those versions and cached in `~/.cache/ng-checkup`. Only `package.json` and `.d.ts` files of those packages are kept.

Installing it in a project

```
npm i -D ng-checkup-cli          # or pnpm add -D / yarn add -D
npx ng-checkup                   # the installed command
```

The package installs two command names, `ng-checkup` and `ng-checkup-cli`, both pointing at the same program. A `"checkup": "ng-checkup"` script in `package.json` therefore works with either.

The package name

Up to 1.5.0 the package was published as `ng-checkup`. From 1.6.1 it is `ng-checkup-cli`; the code is the same and the old versions stay on npm, deprecated with a pointer to the new name. The tool still calls itself `ng-checkup` in reports, baselines, trends and SARIF files, so files written by 1.x keep working.

Exit codes

CODE	MEANING
`0`	The run finished.
`1`	`--fail-on` tripped: an area is graded at or below the threshold, or a measure is worse than the baseline.
`2`	Usage or runtime error: an unknown option, a bad config file, an unknown project, `--fail-on regression` without a baseline, a `--since` ref git cannot resolve.

Areas graded `info` never fail a run.

3. The command line

```
ng-checkup [path | repository] [options]
ng-checkup init [path] [--ci <provider>] [--dry-run] [--force]
ng-checkup baseline [path] [--force]
ng-checkup history [path] [--points <n>] [--report <file>] [--json]
ng-checkup url <address> [--json] [--offline]
ng-checkup fleet <list> [--report <file>] [--online] [--json]
ng-checkup diff <before.json> <after.json> [--report <file>] [--json]
ng-checkup mcp
```

3.1 The main command

Arguments

ARGUMENT	MEANING
`path`	The repository root. Default: the current directory.
`repository`	A remote repository instead of a folder: `github:owner/repo`, `gitlab:owner/repo`, a GitHub or GitLab URL, or any `.git` URL. `#<branch, tag or sha>` picks a revision. It is cloned shallow into a temporary folder and removed afterwards (see section 15).

Options

OPTION	MEANING
`-r, --report <file>`	Write the full report. The format comes from the extension: `.html`, `.md`, `.json`, `.sarif`, `.codequality.json`, `.comment.md`, `.svg` (a README badge), `badge.json` or `x.badge.json` (the badge for shields.io), `.issues.csv` or `.issues.md` (a backlog). Repeatable, so one run can write several files.
`--json`	Print the report as JSON to stdout instead of the summary.
`-v, --verbose`	Print every finding with its files in the terminal.
`--plan`	Add the upgrade plan to the latest Angular, and the migration plan for AngularJS or Primo code.
`-p, --project <name>`	Analyse one project of an Angular CLI or Nx workspace, by its name in `angular.json` or `project.json`.
`--client <name>`	Write the HTML report as a client deliverable (implies `--plan`).
`--prepared-by <text>`	Name on the client report's cover. Default: `$NG_CHECKUP_PREPARED_BY`.
`--notes <file.md>`	Your recommendations, estimates and price for the client report. A missing file is created as a template.
`--history`	Add the last twelve months from git to the client report.
`--trend <file>`	Compare with a trend file: the score, the initial bundle and the measures over the last 30 days.
`--trend-record`	Also add this run to the trend file (meant for the default branch).
`-w, --watch`	Re-run on every change and show how the score and the measures moved since the watch started.
`--fail-on <grade>`	Exit with 1 when any area is graded `poor` (or `fair`), or with `regression` when a measure is worse than the baseline.

OPTION	MEANING
`--since <ref>`	Keep only the findings in files changed since the merge base of `<ref>` and `HEAD`. Grades, score and measures stay the whole repository's.
`--only <ids>`	Report only these areas, comma-separated check ids (`security,testing`).
`--skip <ids>`	Leave these areas out of the report and the score.
`--max-examples <n>`	Example files listed per finding. Default 5.
`--attribute-selectors`	Also judge directives and attribute-selector components, inputs and outputs (see the architecture area).
`--offline`	Nothing online: no npm registry, no osv.dev lookup.
`--config <file>`	Config file. Default: `ng-checkup.config.json` in the root.
`--no-color`	Plain output. Colour is also off when `NO_COLOR` is set or stdout is not a terminal.
`-h, --help`	Show the help.
`--version`	Show the version.

Rules between options

- `--watch` needs a local checkout; it cannot be combined with a remote repository.
- A remote repository has to be cloned, so it cannot be combined with `--offline`.
- `--history` reads the git history, which a shallow clone of a remote repository does not have.
- `--only` and `--skip` take check ids from section 1. An unknown id is a usage error that lists the valid ones.
- `--fail-on regression` needs a baseline file. Without one the run stops with code 2 and says how to create it.

Environment variables

VARIABLE	MEANING
`NO_COLOR`	Turns colour off.
`NG_CHECKUP_PREPARED_BY`	Default for `--prepared-by`.
`NG_CHECKUP_CACHE`	Moves the typings cache from `~/.cache/ng-checkup`.
`XDG_CACHE_HOME`	Used for the cache location when `NG_CHECKUP_CACHE` is not set.

3.2 `ng-checkup init`

Sets a repository up: a script, a config file, a baseline and a CI step. See section 9.

OPTION	MEANING
`--ci <provider>`	`auto` (default), `github`, `gitlab`, `azure`, `bitbucket` or `none`.
`--dry-run`	Show what would change without writing anything.
`--force`	Replace an existing `checkup` script.

3.3 `ng-checkup baseline`

Writes `ng-checkup.baseline.json` with today's measures. See section 7.

OPTION	MEANING
`--force`	Accept measures that went up since the last baseline.

3.4 `ng-checkup history`

How the score and the measures moved over the last months. See section 12.

OPTION	MEANING
`--points <n>`	Months back, 1 to 60. Default 12.
`-r, --report <file>`	Write it as `.html` (with charts), `.md` or `.json`. Repeatable.
`--json`	Print it as JSON.
`--config <file>`	Config file for `ignore`, `only` and `skip`.

3.5 `ng-checkup url`

Which framework a public web app runs, from what a browser downloads. See section 15.

OPTION	MEANING
`--json`	Print the result as JSON.

3.6 `ng-checkup fleet`

One page across many repositories. See section 16.

OPTION	MEANING
`-r, --report <file>`	Write it as `.html`, `.md` or `.json`. Repeatable.
`--online`	Ask the npm registry and osv.dev for each repository (slower).
`--json`	Print it as JSON.

3.7 `ng-checkup mcp`

A Model Context Protocol server on stdin and stdout, until the client closes it. See section 19.

3.8 `ng-checkup diff`

What changed between two reports saved with `--report x.json`, the proof of a piece of work: the score, the grades that moved, the measures that went up or down, and the findings that are new or resolved. It reads the two files and nothing else, so it runs anywhere and on reports of any age, and it exits 0 even when things got worse, since `--fail-on regression` is what gates a build.

```
npx ng-checkup-cli --report before.json
# ... the work ...
npx ng-checkup-cli --report after.json
npx ng-checkup-cli diff before.json after.json --report diff.html
```

OPTION	MEANING
`-r, --report <file>`	Write it as `.html`, `.md` or `.json`. Repeatable.

OPTION	MEANING
`--json`	Print it as JSON.

Findings are matched as a whole, by what they are about, and not file by file, because a report lists only a few example files per finding. What only one report has is neither better nor worse: a measure a newer version added is listed apart, and findings in an area only one report has are not new. The report says so when the two come from different versions of the tool, since a new finding may then come from a new rule, when they name different projects, and when the "after" report is the older one. A `--since` report narrows the findings, so those are not compared at all; the measures still are.

4. The configuration file

`ng-checkup.config.json` in the project root. Every key is optional, and every command-line flag overrides its key.

```
{
  "reports": ["checkup.html"],
  "failOn": "regression",
  "maxExamples": 5,
  "offline": false,
  "ignore": ["src/legacy", "**/generated", "apps/*/e2e"],
  "baseline": "ng-checkup.baseline.json",
  "plan": false,
  "attributeSelectors": false,
  "only": [],
  "skip": ["bundle"]
}
```

KEY	TYPE	MEANING	FLAG
`reports`	string[]	Report files to write, relative to the config file.	`--report`
`failOn`	`"poor"`, `"fair"`, `"regression"`	Exit with 1 at that grade, or when a measure is worse than the baseline.	`--fail-on`
`maxExamples`	number	Example files listed per finding.	`--max-examples`
`offline`	boolean	Nothing online. Wins over the editor's setting too.	`--offline`
`ignore`	string[]	Paths left out of the analysis (see below).	
`baseline`	string	The baseline file, relative to the config file. Default `ng-checkup.baseline.json` in the root.	
`plan`	boolean	Add the upgrade plan to every run.	`--plan`
`attributeSelectors`	boolean	Also judge directives, attribute selectors, inputs and outputs.	`--attribute-selectors`
`only`	string[]	Report only these areas, by check id.	`--only`
`skip`	string[]	Leave these areas out of the report, the score and the baseline.	`--skip`

Validation. An unknown key or a wrong type stops the run with an error that names the key and lists the allowed ones. A typo cannot silently switch off a CI threshold.

`ignore` patterns. A path matches everything below it (`src/legacy`), `*` matches within one folder (`apps/*/e2e`), `**` crosses folders (`**/generated`). An ignored folder is not even walked, which is what makes ignoring a large generated folder cheap. Report files the run writes are ignored automatically, so an HTML report written into the repository is never read back as a template.

Where it is looked for. In the analysed root, or the file `--config` names. For a remote repository, the repository's own config file is applied, but report files it names are not written into the temporary clone.

5. Grades, the score and severities

Grades

Every area gets one grade:

GRADE	MEANING	SCORE POINTS
`good`	The area is current.	100
`fair`	Work to do, nothing urgent.	60
`poor`	Blocks upgrades, out of support, or unsafe.	20
`info`	Nothing to grade, an input was missing, or the area is never graded (translations, accessibility).	not counted

The score

The score is the average of the graded areas' points, rounded: three good areas and one poor score `(100 + 100 + 100 + 20) / 4 = 80`. An area marked `info` is left out of the average entirely. `--only` and `--skip` change which areas are averaged. When nothing is graded, there is no score (shown as `–`).

The score is meant to compare a codebase with itself over time, and two codebases with each other. Because translations and accessibility are never graded, adding them in 2.0 changed no score: a 1.x score and a 2.0 score on the same code are the same number.

Severities

Every finding has one severity:

SEVERITY	MEANING
`high`	Blocks an upgrade, out of support, or a security risk. High findings are lifted into the top priorities.
`medium`	Real work that should be planned.
`low`	Worth knowing; often the older style rather than a defect.
`info`	Context, such as a missing input. Never shown as a problem in SARIF or Code Quality.

Top priorities

The terminal summary, the Markdown report and the HTML report open with up to five priorities: the high findings first, in area order, then medium ones. Each names its area and its first files.

Version-aware defaults

Defaults that changed between Angular and TypeScript versions are applied for the version the project runs:

- A component without `standalone` is standalone from Angular 19, NgModule-declared before.
- A component without `changeDetection` is OnPush from Angular 22, eager before.
- An app without a zone provider is zoneless from Angular 21.
- `strict` is on by default from TypeScript 6; `strictTemplates` is on by default from Angular 22.

The report marks such values as defaults ("default in Angular 22"), so a reader can tell a setting from an inherited default.

6. The areas, one by one

Each area below lists what it reads, how it grades, its findings and how to fix what it reports. The id in parentheses is what the JSON report, `--only` and `--skip` use.

6.1 Primo NDE migration (`primo-nde`)

Only for a repository holding an Ex Libris Primo VE customization package: the AngularJS module `viewCustom` , or components registered under Primo's hook names (`prmSearchBarAfter`). It replaces the AngularJS area: a Primo package is rebuilt in the New Discovery Experience custom module (ExLibrisGroup/customModule, an Angular application), not upgraded in place. NDE is announced as the only Primo UI from January 2028, and VE customizations do not carry over.

Reads: every view folder (a folder holding `js/` ; a network's `...CENTRAL_PACKAGE` is the central package). Per view: each hook once (`prm-search-bar-after`), the helper components and directives the hooks render, `parentCtrl` reads (`parentCtrl.x` , `parentCtrl[...]` , not the `parentCtrl: '<'` binding every hook declares) with the paths read most, `$http` calls, third-party integrations named in the code (BrowZine/LibKey, Altmetric, Unpaywall, Google Analytics/Tag Manager, Matomo, LibChat, LibraryH3lp, HathiTrust, StackMap), and static home pages (`html/home_<lang>.html` , `html/homepage/homepage_<lang>.html`). A `js/custom.js` that only concatenates the view's other files is not counted again, and neither are copies outside the view folders.

Findings: NDE replaces VE (high), the hooks to rebuild, `parentCtrl` reads (where the effort and risk sit, since NDE replaces them with `hostComponent` and the NgRx app state), integrations, the central package, home pages.

Grade: poor when `parentCtrl` is read or more than ten hooks are filled; fair otherwise.

Plan: with `--plan` , four phases: decide what NDE still needs and set up the custom module; branding and home pages; rebuild the hooks, central package first; test in proxy mode and switch view by view.

6.2 AngularJS migration (`angularjs`)

Only when the repository has AngularJS: code, templates, or the `angular` or `@angular/upgrade` package.

Reads: JavaScript and TypeScript. Registrations (`.component` , `.directive` , `.controller` , `.service` , `.factory` , `.provider` , `.filter`) count when their call chain starts at `angular.module(...)` , directly or through a variable assigned from it; a registration on a module from another file counts once the repository shows AngularJS elsewhere, so `Vue.component(...)` in a Vue app never does. Specs, checked-in copies of AngularJS (recognised by their licence header), `vendor` , `bower_components` and minified files are left out. Templates count when they hold `ng-*="..."` attributes, except bootstrap ones (`ng-app`) and the `ng-reflect-*` attributes Angular writes into the DOM. The version comes from `package.json` , the lockfile, `bower.json` , a CDN script tag or a checked-in `angular.js` .

Hybrid apps: `UpgradeModule` , `downgradeModule` or `UpgradeAdapter` from `@angular/upgrade` . "Migrated" is the share of Angular components among all components, directives and controllers; a class registered for AngularJS with `downgradeComponent` or `downgradeInjectable` counts as migrated.

Findings: end of life (high, AngularJS ended in January 2022), a version before 1.8, controllers and `ng-controller` , directives using `compile` , `replace: true` or `terminal: true` (which `UpgradeComponent` cannot wrap), `$scope` , scope events used as a message bus (`$rootScope.$broadcast/$emit/$on` , `$scope.$broadcast/$emit`), runtime `$compile` , `$watch` , `ng-include` , AngularJS packages with their Angular replacement (`angular-route` , `angular-translate` , AngularJS Material, ...), `UpgradeAdapter` , Bower, plain JavaScript, and packages left over without code.

Grade: poor while AngularJS code is left; fair for a hybrid app with at least 80% of its components on Angular, and for leftover packages without code.

Plan: with `--plan`, four phases in the order of the Angular upgrade guide, with this codebase's counts on every item: prepare on AngularJS (1.8, TypeScript, controllers to components, state off `$scope`); run Angular next to AngularJS; move the code bottom up (services, filters, components, templates, packages); remove AngularJS. Items the code does not need are left out.

6.3 Angular version & tooling (`version`)

Reads: the installed `@angular/core` version (the `package.json` range when dependencies are not installed), the latest version on npm (or the newest major this release knows, offline), every `@angular/*` package's major, the build targets in `angular.json` or the Nx `project.json` files, and a short list of end-of-life packages.

GRADE	WHEN
good	on the latest major or one behind
fair	two majors behind; a project built only with a webpack-based builder; mixed `@angular/*` majors; end-of-life tooling (Protractor, TSLint, codelyzer, node-sass)
poor	three or more majors behind (out of support); an end-of-life runtime package (`@angular/http`, `@angular/flex-layout`)

Angular ships a major roughly every six months and supports each for 18 months, so the newest three are supported. `ng update` moves one major at a time, so the gap is also the number of upgrade steps.

Fixing it: `ng update @angular/core@<next> @angular/cli@<next>`, one major at a time. For the builder: `ng update @angular/cli --name use-application-builder`.

6.4 Deprecated APIs (`deprecated-apis`)

Reads: the `@deprecated` tags in the `@angular/*`, `@ngrx/*` and `rxjs` type definitions of the versions the project uses (installed, else the versions the lockfile pins, downloaded once and cached), matched against every symbol the code imports from those packages, plus enum members such as `ChangeDetectionStrategy.Default`. Templates are scanned for `*ngIf`, `*ngFor` and `[ngSwitch]`, which no import reveals. On RxJS 7 and later, `.toPromise()` calls are counted: removed in RxJS 8 in favour of `firstValueFrom` / `lastValueFrom`.

There is no hand-kept list: the report quotes exactly what the installed version marks, in Angular's own words. A function counts as deprecated only when every overload is, and Angular's private `ɵ` names are ignored. The "deprecated two majors ago" rule applies to Angular and NgRx, whose majors move together; RxJS deprecations are reported but never make the area poor on their own.

GRADE	WHEN
good	nothing deprecated in use
fair	deprecated APIs in use, all deprecated less than two majors ago
poor	an API deprecated two or more majors before the installed version, which any upgrade may remove; or an import the next major no longer exports
info	the type definitions could not be read

The next major. The check also downloads the type definitions of the next Angular major for the `@angular/*` and `@ngrx/*` packages the code imports, and compares them with the installed ones:

- an imported name the next major **no longer exports** is a high finding: that import stops compiling after `ng update`. The upgrade plan's first step lists the same names.
- an imported name or enum member the next major **newly deprecates** is a low finding: advance notice, no grade change.

When the project is on the latest major and npm's `next` tag is a prerelease of the one after, that prerelease is compared, so during an Angular release candidate the report already says what the new major breaks. Offline, only typings already in the cache are compared.

Fixing it: each finding quotes Angular's replacement. For templates: `ng generate @angular/core:control-flow`.

6.5 Third-party compatibility (`dependencies`)

Reads: the `peerDependencies['@angular/core']` range of every dependency (from `node_modules`, else from the registry for the version the lockfile pins), so any UI kit, state library or utility is covered without a list of known names. Angular's own packages, NgRx and Nx are left out, since `ng update` and `nx migrate` move them. Online, it asks the registry for each package's releases: the first one that supports each next Angular major, and whether the package is deprecated. Lookups run eight at a time with a four-second timeout each.

GRADE	WHEN
good	every package accepts the installed Angular and all majors up to the latest
fair	a package blocks a later upgrade step but has a release that supports it, or a package is deprecated on npm
poor	a package does not accept the installed Angular (installed with `--force`, `--legacy-peer-deps` or an override), or no release supports the latest Angular
info	dependencies are not installed and the registry was not used

With `--plan`, each package update is scheduled at the step that needs it ("Update ngx-x to 5.0.0" at the step to 19); a package with no release for a major says so at that step.

6.6 Change detection & zoneless readiness (`change-detection`)

Reads: whether the app is zoneless (`provideZonelessChangeDetection()` before Angular 21; the absence of `provideZoneChangeDetection()` from 21 on), each component's `changeDetection`, files that inject `NgZone`, and zone-only test helpers (`fakeAsync`, `tick`, `waitForAsync`, `flush`).

GRADE	WHEN
good	zoneless
fair	zone.js, with at least half of the components on OnPush
poor	zone.js, with less than half on OnPush

Fixing it: move components to OnPush first (each one then notifies Angular through signals, `markForCheck()`, the async pipe or events), then switch to zoneless and rewrite zone-based test helpers with fake timers.

6.7 Components & templates (`components`)

Reads: standalone vs. NgModule-declared components; `input()` / `model()` / `output()` vs. `@Input()` / `@Output()`; built-in control flow (`@if`, `@for`, `@switch`) vs. structural directives; `@defer` blocks; manual `.subscribe()` calls in components and directives; colour literals in stylesheets and inline styles.

Grade: the average adoption of standalone, signal inputs and built-in control flow, counting only what the code uses: 80% or more is good, 40% or more is fair, below that is poor.

Manual subscriptions (low finding, measure `manual-subscriptions`, not graded): every `.subscribe(` inside a `@Component` or `@Directive` class, where `toSignal()` or the async pipe would let Angular own the subscription. A subscription counts as tied to the component's lifetime when `takeUntilDestroyed`, `takeUntil`, `untilDestroyed`, `take` or `first` is in its chain, including through a local variable or local factory initialised with one.

Subscriptions nothing ends (low finding, measure `unended-subscriptions`): a subscription with no lifetime operator in its chain, no `takeWhile`, no cleanup anywhere in the class (`ngOnDestroy`, `unsubscribe()`, `DestroyRef`, `.onDestroy(`, `Subscription.add`), and a source that outlives the component. Left out because they complete or die with the component: `HttpClient` calls, `ActivatedRoute`, a form's `valueChanges` / `statusChanges`, a dialog's `afterClosed()` / `onClose`, `QueryList.changes`, `translate.get()`, `of` / `from` / `forkJoin` / `timer` / `EMPTY`, and a `Subject` or `EventEmitter` the class creates as a field. A service call that completes after one value cannot be told apart from the code, so the count is an upper bound and the finding stays low.

Colour literals outside the design tokens (low finding, measure `hardcoded-colors`): `#fff`, `#ffffff80`, `rgb()` / `rgba()` / `hsl()` / `hsla()` and the common named colours in a colour declaration (`color`, `background`, `border`, `outline`, `fill`, `stroke`, `box-shadow`, `caret`, `text-decoration`, `column-rule`) of any `.css`, `.scss`, `.sass` or `.less` file, and in inline `style="..."` attributes and `[style.x]="'...'"` bindings. A theme (dark mode, a brand per customer, high contrast) changes the tokens; a colour written into a rule does not change with it. Not counted: the value of a custom property (`--brand: #0b5`) or of a SCSS/Less variable (a token definition, wherever it sits), files named for the tokens (`_tokens.scss`, `variables.less`, `theme.css`, `palette.scss`, `colors.scss`, `vars`, `themes`), comments, `transition` / `animation` values, anything inside `url(...)`, minified CSS, vendor folders and test code.

Template speed (four low findings and measures, not graded): what a template makes change detection repeat. A template is read only when a component class in application code owns it, through `templateUrl` or an inline `template`; a template several classes share needs all of them to agree.

- `*ngFor` *without* `trackBy` (`untracked-ngfor`): a list rebuilt from new objects recreates every row. A list of strings or numbers is fine as it is and cannot be told from the template, so the count is an upper bound.
- `@for` *with* `track $index` (`track-by-index`), the variable it was renamed to included: state in the DOM or a child stays at the position when items move. Harmless on a list that never changes order.
- *Method calls in template bindings* (`template-method-calls`): a method the class itself declares, called in an interpolation, property binding, structural directive, block condition or `@let`. Signals are not methods and do not count, nor do event handlers, `track` expressions, calls on another object, template variables, getters or inherited methods. A cheap read costs nothing, so this is a list to look through, not to empty.
- *Impure pipes* (`impure-pipes`): `pure: false` on a pipe this repository declares and a template uses.

A literal array and a field of the class that keeps the same items are left out of both loop rules. On the corpus, per repository: 0 to 80 `*ngFor`, 0 to 12 `track $index`, 0 to 41 method calls, 0 to 2 impure pipes.

Fixing it

```
ng generate @angular/core:standalone
ng generate @angular/core:signal-input-migration
ng generate @angular/core:output-migration
ng generate @angular/core:control-flow
```

For subscriptions: `takeUntilDestroyed()` or `toSignal()`. For colours: `var(--token)` or the variable.

6.8 State management (`state`)

Reads: files importing classic NgRx (`@ngrx/store` , `@ngrx/effects` , `@ngrx/entity` , `@ngrx/router-store` , `@ngrx/component-store`) with their reducers, effects, selectors and action creators; files using `@ngrx/signals` ; `signal()` / `computed()` / `linkedSignal()` calls and `toSignal()` ; injectable services that hold state in a `BehaviorSubject` .

GRADE	WHEN
good	signals in use and no classic NgRx
fair	classic NgRx, or `BehaviorSubject` services without signals
poor	the `@Effect()` decorator, removed in NgRx 15

Classic NgRx is supported, so it never makes the area poor on its own. The area measures the distance to signals, which is what zoneless change detection rewards.

6.9 Architecture (`architecture`)

Reads: route objects (`{ path, component }` is eager, `loadComponent` / `loadChildren` lazy); the import graph and the tags, attributes and pipes templates use (for dead code); file sizes in lines of code; constructors of components, directives, pipes and services; classes implementing a router guard or resolver interface or `HttpInterceptor` ; git history for hotspots.

A `component:` route counts as eager only in a file loaded at startup: statically reachable from the build target's `main.ts` . In a module that is itself behind `loadChildren` , the route loads with that module and counts as lazy.

GRADE	WHEN
good	most routes are lazy and no file is over 1 000 lines
fair	more eager than lazy routes (with five or more routes), or a file over 1 000 lines
poor	ten or more eager routes and none lazy

File size alone never makes the area poor. Constructor injection and class-based guards are low findings that never change the grade: they work, they are only the older style.

Hotspots. A file of at least 300 lines changed in at least 10 commits over the last twelve months (merges left out, limited to the analysed folder) is a hotspot, ranked by lines times commits; the five worst are listed. That is where a refactoring pays off first. Hotspots never change the grade and are not a measure. In a shallow clone there is one commit of history, and the report says so.

Dead code. Low findings with measures, never part of the grade:

- **Files nothing imports** (`unused-files`): application files no other file imports in any way (a value or a type, eagerly, through `import()` , from a spec, through `new Worker(new URL('./x.worker', import.meta.url))` , or an Angular 7 string `loadChildren`), that no build target or tsconfig names, and that are no library entry (`ng-package.json` , a project `package.json` 's `exports` , a tsconfig `paths` target). Only files inside the `sourceRoot` of a project that uses Angular are candidates; `main.ts` , `polyfills.ts` , `test.ts` , environment files, `public-api.ts` , configs, stories and e2e folders are left out by name. It is one level deep on purpose: a file only another unused file imports is listed after the first is deleted.
- **Components and pipes no template uses** (`unused-components` , `unused-pipes`): a component whose tag appears in no template (external, inline or `index.html`) and whose class no file names outside the `declarations` / `imports` / `exports` arrays; a pipe no template calls and no file names. Routes, `loadComponent` , `dialog.open(X)` , `bootstrap` , `inject(XPipe)` and `providers` count as uses. Only plain tag

selectors are judged; selectors with a class or `:not(...)`, declarables without a selector, default exports and library projects never are.

- **Directives and attribute selectors no template matches** (`unused-attribute-selectors`), only with `--attribute-selectors`: a selector part matches when a template has its tag (if it names one) and its attributes, as a plain attribute, a binding, a `*` structural directive or an output binding. `hostDirectives` and queries by class count as uses.
- **Inputs and outputs no template binds** (`unused-inputs`, `unused-outputs`), only with `--attribute-selectors`: by the name a template binds, from decorators, signal functions and metadata arrays; code that sets one by name (`setInput`, `inputBinding`, Ionic `componentProps`) counts. Names are compared across the repository, so the list can miss unused ones but never calls a bound one unused. Component inputs are not judged when the router binds inputs from the URL.

A file or class loaded in a way the analysis cannot see (a path built at runtime) shows up as unused: leave it out with `ignore`.

Fixing it: `loadComponent` per route; `ng generate @angular/core:inject` for constructors; functional guards and interceptors; split large files by responsibility, the largest hotspot first; delete what is unused.

6.10 Testing (`testing`)

Reads: the unit-test runner (from the test targets' builders, else the dependencies: Karma, Jest, Vitest); spec files; which components, services, signal stores and utility files have a spec (an `x.spec.ts` next to `x.ts`, or any spec that imports the file, directly or through a barrel); Jasmine-only calls (`spyOn`, `jasmine.*`); the end-to-end setup (Protractor, Playwright, Cypress).

GRADE	WHEN
good	a current runner and at least 30% of components, services, stores and utilities with a spec
fair	under 30% with a spec
poor	Karma, Protractor, or no specs at all

"Has a spec" is a proxy for coverage, not coverage: it needs no test run.

Test quality (findings and measures, not graded):

- **Tests that assert nothing** (`tests-without-assertions`, medium): an `it` / `test` / `fit` body that calls no `expect`, no `expect*` / `assert*` / `verify*` / `check*` / `should*` helper, no function of the same file that asserts, no namespace assertion (`expect.element`, `expect.soft`, `assert.equal`), no `@ts-expect-error` (a type test asserts at compile time) and no Cypress command. `xit`, `.skip` and `.todo` are not counted.
- **Assertions in a subscribe the test does not wait for** (`unawaited-subscribe-assertions`, low): the `expect` sits in a `.subscribe()` callback and the test takes no `done`, returns nothing (a returned promise, such as `() => new Promise((done) => ...)`, is awaited by the runner), runs in no `fakeAsync` / `waitForAsync` / `async` / `inject` wrapper, awaits nothing and flushes nothing (`flush`, `flushMicrotasks`, `tick`, `expectOne`, `whenStable`, fake-timer calls, `testScheduler.run`). With an asynchronous source the test ends first and passes without asserting.
- **Spec files with `NO_ERRORS_SCHEMA` or `CUSTOM_ELEMENTS_SCHEMA`** (`schema-suppressed-specs`, low): a misspelt input or a component missing from `imports` passes the spec; in `configureTestingModule` neither schema even applies to a standalone component's own template.

Fixing it: Vitest is the Angular CLI's default runner since v21 (`@angular/build:unit-test`); the finding counts the Jasmine-specific calls a move touches. For a subscribe, `expect(await firstValueFrom(source$))` asserts either way.

6.11 Type safety (`type-safety`)

Reads: `strictNullChecks` , `noImplicitAny` and `strict` from the application tsconfig (with `extends` resolved), `strictTemplates` from `angularCompilerOptions` , and the escape hatches: explicit `any` per 1 000 lines (an `any` under an eslint-disable for `no-explicit-any` is reported apart and left out of the count, so the number matches the project's own lint rule), non-null assertions (`x!`), `@ts-ignore` / `@ts-expect-error` , `$any()` in templates, and `UntypedForm*` imports. A strict flag that is off shows on the tsconfig line that sets it, following `extends` .

GRADE	WHEN
good	`strictNullChecks` , `strictTemplates` and `noImplicitAny` on, at most 2 `any` per 1 000 lines
fair	one of them off, or more `any` than that
poor	both `strictNullChecks` and `strictTemplates` off

Fixing it: in a large codebase, stage it: a separate tsconfig that checks one folder at a time, and a count in CI that may only go down (the baseline does exactly that).

6.12 Security (`security`)

Known vulnerabilities. Every direct dependency at its exact version is looked up in OSV (osv.dev, which carries the GitHub advisories for npm). AngularJS from Bower, a CDN or a checked-in copy is looked up as the npm package `angular` . Each package is listed with its CVE ids and the nearest fixed version; dev-only dependencies get a separate low finding. Offline, or when OSV does not answer within eight seconds, the lookup is skipped and the report says "not checked". Not a measure: the count grows when an advisory is published, not when the code gets worse.

Escape hatches in application files: `bypassSecurityTrustHtml` / `Style` / `Script` / `Url` / `ResourceUrl` ; HTML written straight into the DOM (`innerHTML` / `outerHTML` assignments, `insertAdjacentHTML` , `document.write` , `Renderer2.setProperty(el, 'innerHTML', ...)`); strings run as code (`eval` , `new Function`); `[innerHTML]` / `[outerHTML]` bindings.

GRADE	WHEN
good	none of the above apart from `[innerHTML]` bindings (which Angular sanitizes), and no known vulnerability in a runtime dependency
fair	`bypassSecurityTrust*` calls, or a moderate advisory in a runtime dependency
poor	HTML written into the DOM past the sanitizer, `eval` / `new Function` , or a high or critical advisory in a runtime dependency

The check counts escape hatches; it cannot tell a constant from user input, so each finding is a place to review, not a proven hole.

6.13 Translations (`translations`), listed only

Reads: with `@ngx-translate/core` , `@jsverse/transloco` or `@ngneat/transloco` among the dependencies, every JSON bundle named after a language (`en.json` , `de.json` , `pt_BR.json` , `zh-CN.json`) below an `i18n` , `l10n` , `locale(s)` , `translations` or `lang(s)` folder, merged deeply per language. Transloco bundles in a subfolder are scopes and answer under the folder's name (`i18n/admin/en.json` answers `admin.title`).

Keys judged (literal keys only):

- `'a.b' | translate` and `'a.b' | transloco` , either quote, with or without arguments;
- `translate="a.b"` and `[translate]="'a.b'"` ;

- `t('a.b')` inside a `*transloco="let t"` block without `read:`;
- `instant`, `get`, `stream`, `translate`, `selectTranslate` calls on a receiver named like a translation service (`this.translate`, `translateService`, `this.transloco`, `i18n`);
- `marker('a.b')` from ngx-translate-extract.

Not judged: keys built from data (`item.state | translate`, `'status.' + x | translate`), keys in HTML comments, keys in test code including inline test-host templates. Lookups follow ngx-translate's rule that a property may hold a dot in its own name (`"cannot_delete.header"`).

Missing (medium finding, measure `missing-keys`): a key some language has no string for while that language knows the key's first segment, or another language has the key: a typo, a key under the wrong namespace, a string added to one language only. **Unknown namespace** (low finding, no measure): a key whose first segment no bundle of any language has, which a library's own bundle or a backend loader may supply.

Grade: always `info`. The measure sits in the baseline, so `--fail-on regression` fails on a new missing key.

6.14 Accessibility (`accessibility`), listed only

Reads: every template, external or inline (test code left out), with a small tag scanner that understands Angular's attribute spellings (`alt`, `[alt]`, `[attr.alt]`, `attr.alt`) and ignores commented-out markup.

MEASURE	SEVERITY	WHAT	WCAG
`unlabelled-controls`	medium	`<input>`, `<select>`, `<textarea>` with no `<label for>` matching its `id`, no wrapping `<label>`, `<mat-form-field>` or other `*-form-field`, no `aria-label`, `aria-labelledby`, `title` or `placeholder`. Hidden inputs and button-like types are skipped.	1.3.1, 4.1.2
`unnamed-buttons`	medium	`<button>` with no text, interpolation, image with `alt`, projected content (`<ng-content>`), named child or `aria-label` / `aria-labelledby` / `title`	4.1.2
`click-without-keyboard`	medium	`(click)` on `div`, `span`, `li`, `td`, `tr`, `p`, `img`, `i`, `section`, `mat-icon`, headings and other non-interactive elements with no `(keydown)` / `(keyup)` / `(keypress)` handler	2.1.1
`images-without-alt`	medium	`<img>` with no `alt`, unless `aria-hidden="true"` or `role="presentation"` / `none`	1.1.1
`positive-tabindex`	low	`tabindex` above 0	2.4.3

A bound `[id]` beside a bound `[for]` counts as labelled. What a directive or component supplies at runtime is not seen, so the rules err towards silence: an icon ligature (`<mat-icon>delete</mat-icon>`) counts as a button's text.

Grade: always `info`; the measures sit in the baseline.

Fixing it: a `<button>` for anything clickable; `<label for>` or `<mat-form-field>` around every control; `alt` on every image (`alt=""` for decoration); `aria-label` on icon-only buttons; `tabindex` of `0` or `-1` only.

6.15 Circular dependencies (`circular-deps`)

Reads: every static, value-level import between application files, resolved like the Angular CLI does (relative paths, `index.ts` barrels, tsconfig `paths` aliases and `baseUrl`). `import()` and test code are ignored, and so is every import TypeScript erases: `import type`, and a plain import whose names are used only as types, directly or through a barrel. One Angular rule keeps a real cycle visible: constructor parameter types of a decorated class are DI tokens, so they count as runtime references.

Cycles are reported as groups (strongly connected components), each with one concrete loop to cut.

GRADE	WHEN
good	no cycles
fair	up to three groups with ten files in total
poor	more than that

Fixing it: most loops close through a barrel import that drags in a sibling; importing the file directly instead of `./index` usually breaks it.

6.16 Bundle size (`bundle`)

Reads: the stats file of the last `ng build --stats-json` : `browser-stats.json` (Angular 18 and later) or `stats.json` (Angular 17's esbuild builder and webpack builds), in each application's output folder. It does not build anything.

The initial bundle is counted like the Angular CLI's "Initial total": the entry scripts, everything they import statically, and the global styles. The budget is the build target's own `initial` budget from its production configuration, else Angular's default of 500 kB (warning) and 1 MB (error).

GRADE	WHEN
good	under the warning budget
fair	over the warning budget
poor	over the error budget
info	no stats file found

The largest packages in the initial bundle are listed with their size. The stats file's date is in the summary. Not a measure: bundle size grows with every feature; the budget is the limit.

Heavy packages (measure `heavy-packages` , not graded), with or without a stats file: `moment` , whole `lodash` , `rxjs-compat` and `jquery` , each with its lighter way out. Without a stats file a package counts when a file that loads at startup imports it for its code, or when the build target loads it through `scripts` , and the low finding gives no size. A dynamic `import()` , a file only lazy routes reach, a type-only import, `lodash/debounce` , `inject: false` and a package `package.json` lists but nothing imports all stay out. With a stats file the finding is medium and gives the real size in the initial bundle, and also names a listed package only another package pulls in (`lodash` from 50 kB). No saving is given, because it depends on what replaces the package. The measure counts only what the code and the build configuration state, so it reads the same with a stats file and without one. On the corpus: `moment` and `lodash` in airgap-wallet, `lodash` in go-data.

7. The baseline and every measure

A ten-year-old Angular app will not pass `--fail-on poor` on day one, and a CI check that is red from the start gets switched off. The baseline records today's numbers, and CI fails only when one of them gets worse.

```
npx ng-checkup-cli baseline                # writes ng-checkup.baseline.json; commit it
npx ng-checkup-cli --fail-on regression    # in CI: fails only on a number that went up
```

```
Baseline  2 worse
  worse  Type safety: Explicit any in application code 13 → 15
  worse  Type safety: Non-null assertions (x!) 0 → 1
```

The rules

- **Counts, not grades.** Every measure is a count where lower is better.
- **It only goes down.** When numbers drop, the run says so and `baseline` locks the new numbers in. If a number went up, `baseline` refuses and lists it; `--force` accepts it, which is the moment someone has to explain why in review.
- **A plain, sorted JSON file**, so lowering a number is a one-line diff.
- **Projects too.** In a workspace with several projects the file holds each project's numbers beside the workspace's, so a `--project` run gates on its own code. A project's numbers also only go down, even when the workspace total stayed the same because code moved between projects.
- **New measures are never a failure.** A measure the baseline does not know yet (added by a newer release) is reported as new, not as worse. A 1.x baseline keeps working with 2.0.
- **Not measured on purpose:** the number of majors behind (it grows when Angular ships, not when the code gets worse), known vulnerabilities (they grow when an advisory is published), bundle size (it grows with every feature), hotspots (they move with how busy the team is).

Every measure

The key is `<area id>.<measure id>`.

KEY	COUNTS
`primo-nde.parentctrl-reads`	Reads of `parentCtrl` in a Primo VE package
`primo-nde.primo-helper-components`	Helper components of a Primo VE package
`primo-nde.primo-hooks`	Primo hooks filled, per view
`primo-nde.primo-integrations`	Third-party integrations in a Primo VE package
`angularjs.angularjs-components`	AngularJS components, directives and controllers
`angularjs.angularjs-controllers`	AngularJS controllers
`angularjs.angularjs-filters`	AngularJS filters
`angularjs.angularjs-services`	AngularJS services, factories and providers
`angularjs.angularjs-templates`	Templates with AngularJS syntax
`angularjs.scope-files`	Files using `$scope`
`version.end-of-life-packages`	End-of-life packages

KEY	COUNTS
`version.mismatched-angular-packages`	Angular packages on another major
`version.webpack-builders`	Projects on the webpack builder
`deprecated-apis.deprecated-usages`	Deprecated API uses (symbol × file)
`deprecated-apis.legacy-template-directives`	`*ngIf` / `*ngFor` / `[ngSwitch]` uses
`deprecated-apis.to-promise-calls`	`.toPromise()` calls
`dependencies.deprecated-packages`	Packages deprecated on npm
`dependencies.peer-mismatches`	Packages that do not accept the installed Angular
`change-detection.eager-components`	Components with eager change detection
`change-detection.ngzone-files`	Files injecting NgZone
`change-detection.zone-test-helpers`	Zone-based test helper calls
`components.decorator-inputs-outputs`	`@Input()` / `@Output()` decorators
`components.hardcoded-colors`	Colour literals outside the design tokens
`components.impure-pipes`	Impure pipes
`components.legacy-control-flow`	`*ngIf` / `*ngFor` / `[ngSwitch]` uses
`components.manual-subscriptions`	Manual `.subscribe()` in components and directives
`components.module-components`	Components declared in NgModules
`components.template-method-calls`	Method calls in template bindings
`components.track-by-index`	`@for` with `track $index`
`components.unended-subscriptions`	`.subscribe()` in components and directives that nothing ends
`components.untracked-ngfor`	`*ngFor` without `trackBy`
`state.behaviorsubject-services`	Services holding state in a `BehaviorSubject`
`state.classic-ngrx-files`	Files on classic NgRx
`state.effect-decorators`	Removed `@Effect()` decorators
`architecture.class-guards`	Class-based guards and interceptors
`architecture.constructor-injection`	Classes with constructor injection
`architecture.eager-routes`	Eagerly loaded routes (in files loaded at startup)
`architecture.very-large-files`	Files over 1 000 lines
`architecture.unused-files`	Files nothing imports
`architecture.unused-components`	Components no template uses
`architecture.unused-pipes`	Pipes no template uses
`architecture.unused-attribute-selectors`	Directives and attribute selectors no template matches (with `--attribute-selectors`)
`architecture.unused-inputs`	Inputs no template binds (with `--attribute-selectors`)
`architecture.unused-outputs`	Outputs no template binds (with `--attribute-selectors`)
`testing.untested-files`	Components and services without a spec
`testing.tests-without-assertions`	Tests that assert nothing

KEY	COUNTS
`testing.unawaited-subscribe-assertions`	Assertions in a subscribe the test does not wait for
`testing.schema-suppressed-specs`	Spec files with `NO_ERRORS_SCHEMA` or `CUSTOM_ELEMENTS_SCHEMA`
`type-safety.explicit-any`	Explicit `any` in application code
`type-safety.non-null-assertions`	Non-null assertions (`x!`)
`type-safety.strict-flags-off`	`strictNullChecks` / `strictTemplates` / `noImplicitAny` off
`type-safety.template-any-casts`	`$any()` casts in templates
`type-safety.ts-suppressions`	`@ts-ignore` / `@ts-expect-error` comments
`type-safety.untyped-form-files`	Files importing `UntypedForm*`
`security.dynamic-code`	`eval` and `new Function` calls
`security.inner-html-bindings`	`[innerHTML]` bindings
`security.sanitizer-bypasses`	`bypassSecurityTrust*` calls
`security.unsafe-dom-writes`	Direct DOM HTML writes
`translations.missing-keys`	Translation keys that resolve to no string
`accessibility.click-without-keyboard`	Click handlers on non-interactive elements without a key handler
`accessibility.images-without-alt`	Images without alt
`accessibility.positive-tabindex`	`tabindex` above 0
`accessibility.unlabelled-controls`	Form controls without a label
`accessibility.unnamed-buttons`	Buttons without a name
`bundle.heavy-packages`	Heavy packages loaded at startup
`circular-deps.files-in-cycles`	Files in import cycles

8. Reports and output formats

COMMAND	OUTPUT
`npx ng-checkup-cli`	The summary and top five priorities in the terminal.
`--verbose`	Every finding with its files in the terminal.
`--report checkup.html`	One self-contained page: light and dark mode, prints cleanly to PDF, loads nothing from the network.
`--report checkup.md`	Markdown for a pull request, an issue or a wiki.
`--report checkup.json`	Everything, machine-readable. The `Report` type is its schema.
`--report checkup.sarif`	SARIF 2.1.0 for GitHub code scanning and other SARIF viewers.
`--report checkup.codequality.json`	GitLab Code Quality, for the merge request widget.
`--report checkup.comment.md`	A short pull-request comment: what changed against the baseline, a grade per area, the top priorities.
`--report badge.svg`	The score as a README badge: self-contained, in the same bands as the tables.
`--report badge.json`	The same badge as a shields.io endpoint; `x.badge.json` works too, any other `.json` is the full report.
`--report checkup.issues.csv`	A backlog: one row per finding with all its files, for a spreadsheet or the Jira importer.
`--report checkup.issues.md`	The same backlog as a checklist for a GitHub or GitLab issue.
`--json`	The JSON on stdout, for piping.

The terminal summary opens with the tool and version, the workspace name, its kind (Angular CLI, Nx, Primo VE package, AngularJS app, hybrid, custom build), the Angular version, file counts, lines of application code and the run time; then one line per area with its grade and summary; then the score, the baseline comparison when there is one, the trend when there is one, and the top priorities.

The HTML report has a score ring, an overview card per area, the top priorities, every finding with its files and lines, the per-project table of a monorepo, the baseline comparison, the trend, and the upgrade plan. It needs no server and no network.

The JSON report holds: `tool` (name and version), `generatedAt`, `root`, `project` (name, kind, Angular version, file counts, lines of code), `checks` (each with `id`, `title`, `grade`, `summary`, `metrics`, `findings`, `measures`), and when applicable `plan`, `migration`, `projects`, `baseline`, `trend`, `since` and `client`.

SARIF has one rule per area, so alerts survive a rerun. A finding about the whole project (the Angular version, a package that blocks the upgrade) is placed on `package.json`; a strict flag that is off is placed on the tsconfig line that sets it. `info` findings are not reported.

GitLab Code Quality fingerprints ignore numbers and singular/plural, so a finding whose count changes is the same finding to GitLab and the widget shows what a merge request introduces and resolves.

The comment carries a hidden marker so the GitHub Action and the GitLab job edit one comment in place instead of posting a new one per push.

The badge is green from 80, amber from 50, red below and grey when nothing was graded. It is only as fresh as the job that writes it, and the tool commits and publishes nothing, so there are two ways to keep it current: a job with `contents: write` that commits the SVG, or a job that publishes `badge.json` anywhere public and `img.shields.io/endpoint?url=...` in the README, which makes the score public.

The backlog lists, most urgent first, each finding's area, priority (`High`, `Medium`, `Low`), description, every file and a fingerprint that stays the same while the finding does, whatever its count. A run with an export collects every file and caps the other reports again at `--max-examples`. The tool creates no tasks: you import or paste the file. The CSV is UTF-8 without a byte-order mark; `Summary` is the one column Jira's importer requires, and `Description` includes the file list, since that field takes one column. A cell a spreadsheet would run as a formula gets a leading apostrophe, because messages and paths come from the repository being checked, and a cell stops at Excel's limit of 32 767 characters and says how many files it left out. GitHub limits an issue body to 65 536 characters, so a large codebase's checklist may need splitting or `--only`.

9. Setting a project up: `init` and `ng add`

```
ng add ng-checkup-cli        # Angular CLI: installs it and asks which CI to use
npx ng-checkup-cli init      # any workspace, including Nx
```

Both make the same changes and list what they changed:

FILE	CHANGE
`package.json`	A `"checkup": "ng-checkup"` script, and `ng-checkup-cli` in `devDependencies` (`init` only; `ng add` installs it itself). A project that already has `ng-checkup` or `ng-checkup-cli` keeps it.
`ng-checkup.config.json`	Writes `checkup.html` on every run, and an empty `ignore` list.
`.gitignore`	The generated reports, if the repository has a `.gitignore`.
`ng-checkup.baseline.json`	Today's numbers, so CI fails only when one gets worse.
`.github/workflows/ng-checkup.yml`	GitHub Actions: runs on every pull request and on the main branch, uploads the reports (when `.github/` exists).
`ng-checkup.gitlab-ci.yml`	GitLab: includes the template at the `v2` tag and sets `NG_CHECKUP_FAIL_ON`; add `include: [{ local: ng-checkup.gitlab-ci.yml }]` to `.gitlab-ci.yml`.
`ng-checkup.azure-pipelines.yml`	Azure Pipelines, beside the main pipeline: publishes the SARIF report as `CodeAnalysisLogs` for the Scans tab and the HTML report as an artifact.
`bitbucket-pipelines.yml`	Bitbucket: a pull-request step and a main-branch step with the reports as artifacts. When the file exists, it is left alone and the step is printed to paste.

Detecting the CI (`--ci auto`): GitHub when `.github/` exists, GitLab when `.gitlab-ci.yml` does, Azure DevOps when `azure-pipelines.yml` or `.azure-pipelines` does, Bitbucket when `bitbucket-pipelines.yml` does, else none.

Fail-on. With a baseline (existing or the one `init` writes), the CI step runs `--fail-on regression`; otherwise `--fail-on poor`.

Package manager. Detected from the lockfile: pnpm (`pnpm-lock.yaml`), yarn (`yarn.lock`), else npm. The CI steps install with the frozen-lockfile form and run the installed binary.

Idempotent and safe. Running it again changes nothing. It never overwrites a file you wrote: an existing workflow or config is kept (and a note says when an existing workflow still runs `--fail-on poor` although a baseline exists), and a different `checkup` script is kept unless you pass `--force`. `--dry-run` shows the changes and writes nothing. In a folder that does not look like an Angular workspace it says so and still plans the rest.

10. Continuous integration

The same tool runs everywhere: a pull or merge request starts a job on a runner, the job runs `npx ng-checkup-cli` from npm without installing the project, and the results come back where the review happens. With `fail-on` the job turns red, which blocks the merge when the branch requires the check.

On an existing codebase, start with a baseline and `--fail-on regression`. `--fail-on poor` and `--fail-on fair` are for codebases already in good shape.

10.1 GitHub Action

```yaml
name: ng-checkup
on:
  pull_request:
  push:
    branches: [main]        # the default branch records the trend
permissions:
  contents: read
  pull-requests: write      # the comment
  security-events: write    # only with sarif: true
jobs:
  checkup:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: stefankuburovic/ng-checkup@v2
        with:
          fail-on: regression
          sarif: true
```

Inputs

INPUT	DEFAULT	MEANING
`path`	`.`	Folder to check, relative to the repository root.
`fail-on`	empty	`poor`, `fair` or `regression`. Empty never fails.
`plan`	`false`	Add the upgrade plan to the reports.
`comment`	`true`	One pull-request comment, edited in place on every push. Needs `pull-requests: write`; skipped with a warning on a fork pull request.
`sarif`	`false`	Upload findings to GitHub code scanning. Needs `security-events: write`.
`artifact`	`true`	Upload the HTML, Markdown, JSON (and SARIF) reports as the artifact `ng-checkup`.
`offline`	`false`	Skip the npm registry.
`trend`	`true`	Keep a trend in the Actions cache; pull requests show the last 30 days of the default branch.
`args`	empty	Further options, such as `--project admin --max-examples 20`.
`version`	empty	The ng-checkup version to run. Empty runs the version this action was released with.
`node-version`	`22`	Node.js version (22 or newer).
`token`	`github.token`	Token for the comment.

Outputs: `score` (0 to 100, empty when nothing was graded) and `exit-code` (0, 1 or 2).

The Action also writes the job summary. No `npm ci` is needed: versions come from the lockfile. For `version`, 1.6.1 and later run `ng-checkup-cli`; an older version runs the old package name `ng-checkup`.

Tags. `@v2` is the newest 2.x release; `@v1` stays on 1.6.1; `@v2.1.0` pins one release.

10.2 GitHub code scanning without the Action

```
permissions:
  security-events: write
steps:
  - uses: actions/checkout@v4
  - run: npm ci
  - run: npx ng-checkup-cli --offline --max-examples 1000 --report ng-checkup.sarif
  - uses: github/codeql-action/upload-sarif@v3
    if: always()
    with:
      sarif_file: ng-checkup.sarif
      category: ng-checkup
```

Code scanning is free on public repositories and needs GitHub Advanced Security on private ones.

10.3 GitLab CI/CD component

```
include:
  - component: $CI_SERVER_FQDN/stefankuburovic/ng-checkup/ng-checkup@2
    inputs:
      fail-on: regression
```

INPUT	DEFAULT	MEANING
`job-name`	`ng-checkup`	Name of the job, for a second checkup in the same pipeline (one per app of a monorepo). Also the report folder.
`stage`	`test`	Stage the job runs in.
`path`	`.`	Folder to check.
`fail-on`	empty	`poor`, `fair` or `regression`.
`args`	empty	Further options, such as `--plan --project admin`.
`comment`	`true`	Write the merge request comment when `NG_CHECKUP_TOKEN` is set.
`version`	`2.1.0`	The ng-checkup release to run; each component release pins its own.
`trend`	`true`	Keep a trend in the CI cache; merge requests show the last 30 days.
`image`	`node:22`	Node.js image (Node 22 or newer).

The job runs on merge requests and on the default branch, which is what the Code Quality widget compares with. It produces the widget (what the merge request introduces and resolves, on every tier, self-managed included), the reports as artifacts, and one comment per merge request edited on every push.

The comment token. Add a token with the `api` scope as the CI/CD variable `NG_CHECKUP_TOKEN`, **masked but not protected** (merge request branches are not protected, so a protected variable never reaches them). A project access token comments as a bot; on gitlab.com Free, where those need Premium, a personal access token works and comments as you. Without the variable the comment is skipped and the rest runs.

10.4 GitLab template (self-managed GitLab)

A self-managed GitLab cannot include components from gitlab.com:

```
include:
  - remote: https://raw.githubusercontent.com/stefankuburovic/ng-checkup/v2/gitlab/ng-checkup.yml

ng-checkup:
  variables:
    NG_CHECKUP_FAIL_ON: regression
```

Variables: `NG_CHECKUP_FAIL_ON`, `NG_CHECKUP_PATH`, `NG_CHECKUP_ARGS`, `NG_CHECKUP_VERSION`, `NG_CHECKUP_TREND`, `NG_CHECKUP_TREND_POLICY`, `NG_CHECKUP_TOKEN`. If the GitLab cannot reach GitHub either, copy `gitlab/ng-checkup.yml` into the repository and include it with `local:`.

10.5 Azure DevOps

`npx ng-checkup-cli init --ci azure` writes `ng-checkup.azure-pipelines.yml`, a pipeline of its own beside the repository's main pipeline. It installs with the frozen lockfile, runs the checkup with Markdown, HTML and SARIF reports, publishes the SARIF report as the `CodeAnalysisLogs` artifact (the name Microsoft's SARIF SAST Scans Tab extension reads, so findings show in the run's Scans tab when the extension is installed) and the HTML report as the artifact `ng-checkup`.

To run it: Pipelines, New pipeline, Existing Azure Pipelines YAML file, pick the file; then add it as a build validation in the target branch's policy so it runs on every pull request (Azure Repos takes pull-request triggers from branch policies, not from YAML).

10.6 Bitbucket Pipelines

`npx ng-checkup-cli init --ci bitbucket` writes `bitbucket-pipelines.yml` with a step for every pull request and one for `main`: `node:22`, the node cache, install, the checkup with `--fail-on`, and the reports as artifacts. Bitbucket reads one pipelines file and has no include, so when the file exists, `init` leaves it as it is and prints the step to paste under `pipelines: pull-requests:`.

10.7 Any other CI

```
npm ci
npx ng-checkup-cli --fail-on regression --report checkup.md --report checkup.html
```

Keep `checkup.*` as artifacts. Exit code 1 fails the job.

11. Only what a branch changed: `--since`

```
npx ng-checkup-cli --since origin/main --report checkup.comment.md
```

Keeps only the findings that name a file changed since the merge base of `origin/main` and `HEAD`: committed, uncommitted and untracked files. That is the list a pull-request review wants, without the backlog the repository already had.

What stays the whole repository's. The analysis still reads everything, since a cycle, an unused file or a missing key only shows against the rest. Grades, the score and every measure stay the whole repository's, so `--fail-on regression` means exactly the same with and without `--since`.

How findings narrow. Each finding keeps only its files that changed; a finding with none left, or with no files at all (the Angular version, the test runner), is left out. The finding's message keeps the repository's count; its file list and `count` are the changed files. Examples are uncapped during the run and capped again after narrowing, so no changed file is lost to `--max-examples`.

Every format says so: "Findings limited to the 12 files changed since origin/main (3f2a91c0b7de); grades, score and measures are the whole repository's."

The merge base, not the ref. Commits that landed on the target branch after this branch forked do not count as this branch's changes, which is what code hosts compare with too.

In CI the target branch must be fetched: `actions/checkout` with `fetch-depth: 0`, or `git fetch origin main`. A shallow clone has no merge base, and the run stops with a message saying what to fetch (exit code 2).

12. Watch mode, history and the trend

Watch mode

```
npx ng-checkup-cli --watch
```

Re-runs on every save and prints, under the usual summary, how the score and every measure moved since the watch started:

```
Score 72 -> 76 (+4 since the watch started)
  down Explicit any: 41 -> 37
  down Components without OnPush: 12 -> 11
```

Only changed files are parsed again (each file's facts are cached by modification time and size), and the registry is asked again only when `package.json`, a lockfile or `node_modules` changes. Report files the run writes are ignored, so writing them never triggers the next run.

History

```
npx ng-checkup-cli history --report history.html
```

Takes the last commit of each month from git (12 months by default, `--points` up to 60; a month without a commit is skipped), extracts it into a temporary folder with `git archive`, runs the checkup offline on it, and ends with the working tree as it is, uncommitted changes included. The checkout is never touched. The terminal prints one line per point (date, commit, Angular version, score, lines of code) and the measures that moved most; the HTML report has a chart of the score and one per measure that moved most. It needs the full history: a shallow clone stops with a message.

The trend

Every CI run on the default branch can add a point (the score, every measure, and the initial bundle when a build with `--stats-json` ran first) to a small file kept in the CI cache; pull and merge requests then show where the default branch has been heading:

```
Score 72, +4 since 2026-09-23 ▂▃▅▇ · initial bundle 812 kB, +23 kB
```

On by default in the Action and in both GitLab integrations. By hand: `--trend <file.json>` to compare, plus `--trend-record` on the default branch to add the run. Unlike `history`, which rebuilds the past from git, the trend follows the bundle too, from the first recorded run.

13. The upgrade plan and the migration plans

```
npx ng-checkup-cli --plan --report plan.md
```

`--plan` adds the way from the installed Angular to the latest, one major at a time as `ng update` requires:

```
Upgrade plan  12 → 22, 10 steps, one major at a time
  Before you start
    · Move from TSLint to ESLint  ng add @angular-eslint/schematics
  → 13  ng update @angular/core@13 @angular/cli@13
        Node.js ^12.20.0 || ^14.15.0 || >=16.10.0 · TypeScript >=4.4.2 <4.7 (now 4.3.5)
  → 16  ng update @angular/core@16 @angular/cli@16
        · Convert NgModule components to standalone (18)  ng generate @angular/core:standalone
  → 17  ng update @angular/core@17 @angular/cli@17
        · Move templates to built-in control flow (13)  ng generate @angular/core:control-flow
```

- **Before you start:** what has to happen first (imports the next major no longer exports, APIs deprecated two or more majors ago, TSLint leftovers, import cycles).
- **Per step:** the `ng update` command (with Material and NgRx when used), the Node.js, TypeScript, RxJS and zone.js ranges that major requires (a range the project is outside of today is flagged at the step that must move it), package updates scheduled where they are needed, and the migrations this code calls for (standalone, built-in control flow, the esbuild builder, signal inputs, Vitest, zoneless) at the first step where Angular ships them, with the number of places each touches.
- **No time estimate.** Counts are measured; days would be a guess. The client report has an estimate column for your own numbers.

The version data comes from a snapshot of the registry bundled with each release, refreshed before every release.

For AngularJS and Primo VE code the plan is the migration plan of section 6.1 or 6.2. A hybrid app on an old Angular gets both.

14. Monorepos and projects

In an Angular CLI workspace with several projects, or an Nx workspace, the report adds a table with one row per project, worst score first:

```
Projects  5, worst first, graded on their own code
   65  products            apps/products            poor: Change detection & zoneless readiness,
Components & templates, Type safety
   80  products-home-page  libs/products/home-page  poor: Type safety  fair: Components & templates,
State management
```

Each project is graded on the files it owns (a nested library keeps its own) with its own tsconfig. The Angular version and the third-party packages are shared, so they are graded once, for the workspace. Projects that import nothing from Angular (React apps, Node services, Cypress folders) get no row.

`--project <name>` narrows the whole report to one project, which is how a team owning one app runs it in CI. An unknown name is a usage error that lists the names. With several projects, `baseline` records each project's numbers, and `--project admin --fail-on regression` fails only on admin's own numbers.

An Angular app in a subfolder. When the root is no Angular workspace (a Python or Java server with its UI in `server/static`), and exactly one subfolder has an `angular.json`, the workspace is read there and the header says `Angular CLI in server/static`. With several such subfolders the report names them; run it on the one you mean.

15. Remote repositories and live sites

Remote repositories

```
npx ng-checkup-cli github:owner/repo --report checkup.html
npx ng-checkup-cli gitlab:group/app
npx ng-checkup-cli https://gitlab.com/group/app#release-2.4
npx ng-checkup-cli https://example.com/team/app.git#3f2a91c
```

The repository is cloned shallow (only the one commit) into a temporary folder, analysed, and deleted. A `#ref` picks a branch, tag or commit. Nothing from the repository is installed or run. `GIT_TERMINAL_PROMPT=0` makes a private repository fail at once instead of waiting for a password. The report names the codebase by repository and commit, never by the temporary path.

Live sites

```
npx ng-checkup-cli url https://app.example.com
```

Which framework a public web app runs, without its code: the page and the scripts of the same site it loads, including the chunks they import, up to 12 requests. It recognises AngularJS and Angular with their versions (from the licence header, the version object, `new Version("...")` in the bundle, and `ng-version`), whether zone.js is loaded, and React, Vue, Next.js and Nuxt, also several at once. Third-party scripts are skipped (a chat widget built with React does not make a site a React app), except AngularJS from a public CDN. It reads only what any browser downloads: nothing is submitted, nothing behind a sign-in is reached. `--json` prints the result for scripts.

16. Many repositories: `fleet`

```
npx ng-checkup-cli fleet repos.txt --report fleet.html
```

The list file. One repository per line: a local path (relative to the list file) or a remote repository as the main command takes it. Blank lines and `#` comments are skipped; a `#ref` at the end of a remote entry is kept.

```
# front ends
../shop
../admin            # the back office
github:acme/portal#release
```

What it does. Every entry is checked one after the other, as a run of its own would check it: a local folder as it is, a remote one cloned shallow and deleted. The page ranks them worst score first, with the Angular version, the score, the number of high findings, the lines of code and a grade per area. An entry that cannot be checked (a refused clone, a missing folder) is a row saying why, at the end, and the run goes on.

Offline by default, since across many repositories the registry lookups are most of the time and the comparison is between them; `--online` asks the registry and osv.dev for each.

Formats: the terminal ranking, `.html` (one page in the report's styles, a pill per area), `.md` (a table for a wiki), `.json` . Progress goes to stderr, so `--json` on stdout stays parseable. The exit code is 2 only when no entry could be checked.

17. The client report

For consultants and tech leads who hand the result to someone else:

```
npx ng-checkup-cli --client "Acme GmbH" --prepared-by "Jane Doe" --notes acme.md --history --report
acme-health-check.html
```

- **For management, in front:** a cover, a summary in sentences ("Angular 17.1.1, 5 majors behind 22. Of 10 graded areas, 3 are poor..."), the key figures, a table of what to act on with the number of places each touches, and the roadmap one major at a time.
- **Your recommendations:** `--notes` is a Markdown file for what the tool cannot know: your summary, estimates and price. The first run creates it as a template, prefilled with one row per upgrade step and an empty estimate column. Guidance in the template sits in HTML comments and never reaches the client; a section left empty is left out.
- **How the code got here,** with `--history`: the score and the measures that moved most over the last twelve months, as charts.
- **Where to start:** known vulnerabilities and what the next major removes in the key figures, and the hotspots as the place a refactoring pays off first.
- **The full report as an appendix.**
- **Print to PDF** from the browser: the cover, the recommendations and the appendix each start on a new page.

`--client` implies `--plan`. `NG_CHECKUP_PREPARED_BY` sets the name for every run. Nothing in the generated part is an estimate: numbers are measurements, and the estimate column is yours.

18. The VS Code extension

`stefankuburovic.ng-checkup-vscode`, on the VS Code Marketplace and on Open VSX (for Cursor and VSCodium). Version 0.5.0 bundles ng-checkup 2.1.0.

- Every finding at its file and line in the Problems panel; findings about the project as a whole sit on `package.json`, a strict flag on its tsconfig line.
- Why it matters on hover.
- The score in the status bar; click it for the HTML report.
- Runs on open and after each save, re-parsing only the files that changed, in a worker thread so the editor never freezes.
- Reads `ng-checkup.config.json` (`ignore`, `only`, `skip`, `offline`, `attributeSelectors`).

Commands: `ng-checkup: Run`, `ng-checkup: Open report`, `ng-checkup: Open report in browser`.

Settings

SETTING	DEFAULT	MEANING
`ng-checkup.registryLookup`	`false`	Ask the npm registry for the latest Angular and installed packages' metadata. Off: nothing leaves the machine.
`ng-checkup.highSeverity`	`warning`	How high findings show in the editor. Medium findings are warnings, low ones information.
`ng-checkup.maxExamples`	`1000`	Locations each finding lists; the editor wants all of them.
`ng-checkup.runOnSave`	`true`	Analyse again after a TypeScript file, a template, `package.json`, a tsconfig or the config file is saved.
`ng-checkup.debounceMs`	`1500`	Wait after the last save before analysing; saves during a run fold into one follow-up run.
`ng-checkup.attributeSelectors`	`false`	Also show unused directives, attribute selectors, inputs and outputs.

19. The MCP server for editor agents

```
npx -y ng-checkup-cli mcp
```

Serves the checkup over the Model Context Protocol on stdio, so Claude Code, Cursor, Copilot or any MCP client can ask about a workspace directly.

Registering it. In Claude Code:

```
claude mcp add ng-checkup -- npx -y ng-checkup-cli mcp
```

In other clients, register a stdio server with the command `npx` and the arguments `-y ng-checkup-cli mcp`.

Tools

TOOL	ARGUMENTS	RETURNS
`checkup`	`path`, `only` (area ids), `project`, `since` (a git ref), `online`	The Markdown report.
`findings`	`path`, `check` (one area), `severity` (minimum), `file` (one file, relative to `path`), `online`	JSON: `{ root, findings: [{ check, severity, message, detail, files, count }] }`.
`upgrade_plan`	`path`, `online`	The upgrade or migration plan as Markdown, or a sentence saying there is nothing to plan.

`path` is absolute or relative to where the server was started; default that folder.

Behaviour. Runs are offline unless a call passes `online: true`. Findings list up to 50 locations each. The server keeps one analysis cache per folder, so a second question re-parses only what changed. Calls are answered one at a time, in order. A failed analysis (an unknown project, a bad ref) is a tool result with `isError: true` the agent can read, not a protocol error; an unknown method returns JSON-RPC error `-32601`. Nothing but protocol messages is written to stdout.

Protocol. Newline-delimited JSON-RPC 2.0 with `initialize` (protocol version echoed, `2025-06-18` by default), `notifications/initialized`, `ping`, `tools/list` and `tools/call`. Written against the protocol directly, so the package gains no dependency.

20. The programmatic API

```
import { writeFile } from 'node:fs/promises';
import { runCheckup, renderMarkdown, overallScore } from 'ng-checkup-cli';

const report = await runCheckup('path/to/app', { offline: true });

console.log(overallScore(report)); // e.g. 45
for (const check of report.checks) {
  console.log(check.id, check.grade, check.summary);
}

await writeFile('checkup.md', renderMarkdown(report));
```

The package is ESM and ships its type definitions.

`runCheckup(root, options)`

Runs the analysis and returns a `Report`. It never writes anything.

OPTION	TYPE	MEANING
`offline`	boolean	Nothing online.
`maxExamples`	number	Example locations per finding (default 5).
`ignore`	string[]	Paths to leave out.
`plan`	boolean	Add the upgrade plan.
`attributeSelectors`	boolean	Judge directives, attribute selectors, inputs and outputs.
`only` / `skip`	string[]	Areas to keep or leave out. Throws `UnknownCheckError` for an unknown id.
`project`	string	One project. Throws `ProjectNotFoundError`, listing the names.
`since`	string	Narrow the findings to files changed since the merge base. Throws `SinceError`.
`compareNextMajor`	boolean	Compare with the next major's typings (default true).
`cache`	`AnalysisCache`	From `createAnalysisCache()`, to make repeated runs incremental.

Exports

EXPORT	WHAT
`runCheckup`, `buildContext`	The analysis.
`renderHtml`, `renderMarkdown`, `renderSarif`, `renderCodeQuality`, `renderComment`, `COMMENT_MARKER`	The report formats.
`renderBadgeSvg`, `renderBadgeJson`, `renderIssuesCsv`, `renderIssuesMarkdown`	The badge and the backlog; pass a high `maxExamples` for a backlog with every file.
`overallScore`, `topPriorities`	The score and the priority list.
`createAnalysisCache`, `AnalysisCache`	Incremental runs.
`loadConfig`, `CONFIG_FILE`, `CheckupConfig`	The config file.
`planInit`, `InitPlan`, `InitChange`, `InitOptions`	What `init` would change, without writing it.

EXPORT	WHAT
`buildPlan`, `UpgradePlan`, `UpgradeStep`, `PlanItem`, `PlanRequirement`	The upgrade plan.
`buildMigrationPlan`, `buildPrimoPlan`, `MigrationPlan`, `MigrationPhase`	The migration plans.
`collectHistory`, `movedMeasures`, `HistoryReport`, `HistoryPoint`, `HistoryUnavailableError`, `renderHistoryHtml`, `renderHistoryMarkdown`	History.
`createBaseline`, `compareWithBaseline`, `baselineMeasures`, `readBaseline`, `serializeBaseline`	The baseline.
`CHECKS`, `CHECK_IDS`, `checkFilter`	The checks and their ids.
`ANGULAR_VERSIONS`, `BUNDLED_LATEST_ANGULAR_MAJOR`, `fetchLatestAngularMajor`	Version data.
`TOOL`	The tool's name and version.
`SinceError`, `SinceInfo`, `UnknownCheckError`, `ProjectNotFoundError`	Errors.
`Report`, `CheckResult`, `Finding`, `FileRef`, `Grade`, `Severity`, `Metric`, `Measure`, `CheckupOptions`, …	The types. `Report` is the schema of the JSON output.

21. What leaves your machine

Package names and versions, to two public services, and nothing else:

- **The npm registry:** the latest `@angular/core` version (three-second timeout); for every third-party package that depends on Angular, its public metadata (eight at a time, four-second timeout each); the published tarballs of the `@angular/*`, `@ngrx/*` and `rxjs` versions the code imports when `node_modules` is missing, and of the next Angular major's packages it imports. Of each tarball only `package.json` and `.d.ts` files are kept, in `~/.cache/ng-checkup`.
- **osv.dev:** the direct dependencies with their exact versions, to look up known vulnerabilities (eight-second timeout).

`--offline` skips all of it and uses only what is cached; the report then compares against the newest major the release knows and says so. A remote repository argument also clones it from its host; `url` downloads what a browser would. No source, no file names, no telemetry. The tool writes only the reports you ask for, and the typings cache.

22. How it works inside

The pipeline

1. **One walk.** `collectFiles` walks the folder once and sorts what it finds: TypeScript (no `.d.ts`), HTML, JavaScript (outside vendor folders, without minified builds or compiler output beside a `.ts`), `project.json`, nested `angular.json`, translation bundles and stylesheets. `node_modules`, `dist`, `coverage`, `out-tsc`, `tmp`, `bazel-out` and hidden folders are skipped; files over 1 MB are treated as generated. Ignored folders are not entered.
2. **The workspace.** `angular.json` or the Nx `project.json` files give the projects, their roots and `sourceRoot`, build targets, tsconfigs, budgets and output folders; `package.json` and the lockfile give the versions.
3. **Facts per file.** Every TypeScript file is parsed once with the TypeScript compiler API, without a type checker and without building the project. One scan collects imports, components, directives, pipes and their bindings, calls resolved through their imports (a local helper called `signal` is not Angular's), subscriptions, routes, the escape hatches, translation keys and test-quality facts. Templates (external and inline) give their tags, attributes, pipes, legacy directives, translation keys, inline colours and accessibility facts; stylesheets give their colour literals. Facts are cached per file by modification time and size.
4. **Derived facts.** The import graph (relative paths, barrels, `paths`, `baseUrl`), files statically reachable from the entry, files nothing imports, declarables nothing uses, the deprecation index from the typings, third-party compatibility, vulnerabilities, the next major's changes, translation lookups, churn from git and the bundle stats.
5. **Checks.** Each area is a pure function of that context: no check reads the disk, which is what keeps every result reproducible and every check testable with a hand-built context.
6. **Reports.** Renderers turn the `Report` into each format. A check never imports a renderer and a renderer never imports a check.

Principles

- **Syntax only.** No type checker and no build: seconds on a large monorepo, and it works on code that does not compile.
- **Deprecations come from the versions you run.** No hand-kept list; Angular's own `@deprecated` tags, following its re-exports between bundled `.d.ts` chunks.
- **Lines of code are lines that hold code.** Blank and comment-only lines do not count, so documentation does not make a file "large".
- **Generated code is left out of the counts** (a leading `@generated` or "do not edit" comment, names like `api.generated.ts`, a `generated` folder) but stays in the import graph.
- **Cycles are counted as groups,** since individual cycles grow combinatorially.
- **Err towards silence.** A rule that cannot be sure stays quiet; uncertain rules are low severity or opt-in.

23. Quality assurance: tests and the corpus

- **Unit and integration tests:** 248 Vitest tests in 38 files, run against fixture apps and hand-built temporary repositories, including the built CLI end to end. `prepublishOnly` runs the typecheck and the full suite before every publish.
- **The corpus:** `npm run corpus` runs ng-checkup on 19 public repositories pinned to one commit each (Angular 12 to 22, AngularJS and hybrid apps, two Primo VE packages, CLI and Nx workspaces, Ionic, apps up to 180 000 lines) and compares every grade, measure and finding with committed snapshots. CI runs it on every push. A rule change that moves any number shows up as a diff, is read, and is only accepted with `--update` when every changed line is an improvement. The repositories are fetched shallow and their dependencies are never installed.
- **Hand verification.** Every new rule in 2.0 was checked against real hits: the five missing translation keys in airgap-wallet, the label pointing at an input without an `id` in a storefront, assertions inside a `timer()` subscription that never ran, and a private 1 669-file production app whose own guards already hold the same numbers at zero.

24. Versions, tags and the release process

THING	RULE
npm	`ng-checkup-cli@<x.y.z>`; `latest` is the newest release.
Git tags	`v<x.y.z>` for each release, annotated.
Action	The moving major tag (`v2`) points at the newest 2.x; `v1` stays on 1.6.1; a new major gets a new moving tag, so nobody on `@v1` receives a major they did not ask for.
GitLab component	Semver tags (`2.1.0`); `@2` resolves to the newest 2.x. Each component release pins the matching ng-checkup version.
Extension	Its own versions (0.5.0 bundles 2.1.0).

The release order (each step only after the previous one is visible):

1. Refresh the bundled Angular versions (`node scripts/update-angular-versions.mjs`), bump the version, write the CHANGELOG entry, set `NG_CHECKUP_VERSION` in `gitlab/ng-checkup.yml`, commit, tag `v<x.y.z>`, push.
2. Publish: a GitHub release for the tag runs `.github/workflows/release.yml`, which publishes through npm trusted publishing with provenance (once a trusted publisher is configured on npmjs.com); by hand, `npm publish --auth-type=web`.
3. Move the major tag only after the tarball answers 200, gated in the same command, because a tag that names an unpublished version breaks every workflow on it.
4. Bump the GitLab component's `version` default, commit, tag the same number.
5. Bump the extension, package, publish to the Marketplace and Open VSX, release on GitHub with the `.vsix`.

When Angular ships a new major, `docs/new-angular-major.md` is the checklist: refresh the version data, run the tests and the corpus, release as a minor version.

25. Troubleshooting and FAQ

Does it change my code? No. It only reads. The fixes it suggests are Angular's own `ng generate` migrations, which you run yourself.

Deprecated APIs is "info". The type definitions could not be read: dependencies are not installed and the run was offline (or the registry was unreachable) before they were ever cached. Run once online, or install the dependencies.

My Angular 12 app has deprecated APIs "good". Correct for Angular 12: nothing *Angular 12* deprecates is in use. Each upgrade step deprecates more; the version area shows that gap, and the next-major comparison shows what the next step breaks.

Bundle size is "info". It reads the stats of your last build instead of running one. Run `ng build --stats-json`, then the checkup.

Third-party compatibility is "info". Dependencies are not installed and the run was offline. Run online, or install.

"--fail-on regression needs a baseline". Run `npx ng-checkup-cli baseline` and commit the file.

`baseline` refuses. A measure went up since the last baseline. Fix it, or accept it with `baseline --force` and say why in review.

`--since` fails in CI. The runner has a shallow clone without the target branch. Use `fetch-depth: 0`, or `git fetch origin main`.

`history` fails. It needs the full git history; a shallow clone has one commit.

A file is reported as unused but is loaded. It is loaded in a way the analysis cannot see (a path built at runtime). Add it to `ignore`.

A translation key is reported under an unknown namespace. It comes from a library's bundle or a backend loader, or it is missing everywhere. Those are listed but never measured.

An accessibility finding is wrong. The element gets its name or label at runtime from a directive or component. The rules only read markup.

The score is lower than expected. Look at the poor areas first; `--verbose` lists every finding with its files. `--skip` leaves an area out if it does not apply.

Is "with a spec" test coverage? No: it counts files with a spec next to them or imported by one.

Does it support Nx? Yes: `project.json` targets, tsconfig `paths`, a row per project, `--project`.

Does it need RxJS or NgRx? No. Checks report what they find; other libraries are covered through their Angular peer ranges.

Which Angular versions? Angular 2 and newer, and AngularJS 1.x on its way there.

The registry or osv.dev is slow or blocked. Every lookup has a timeout; a timed-out lookup is reported as not checked. `--offline` avoids the network entirely.

A remote repository fails to clone. Private repositories are not prompted for (`GIT_TERMINAL_PROMPT=0`); clone it yourself and run on the folder.

The old command `npx ng-checkup` still runs 1.5.0. That is the old package name. Use `npx ng-checkup-cli`.

26. What changed in 2.1 and 2.0

2.1.0 changes no score or grade: the new checks are findings and baseline measures outside the grade, and a baseline from 2.0 keeps working, with the five new measures counted as not known yet.

- **Template speed** in components: `*ngFor` without `trackBy`, `track $index`, method calls in bindings, impure pipes.
- **Heavy packages** in bundle: `moment`, whole `lodash`, `rxjs-compat`, `jquery`.
- **`ng-checkup diff`:** what changed between two saved reports.
- **A README badge:** `--report badge.svg` and `badge.json` for shields.io.
- **A backlog:** `--report x.issues.csv` and `x.issues.md`, one item per finding with all its files.

2.0.0 is a major release because the report grows: two new areas and new measures, which a consumer counting areas or reading the JSON by position notices. Nothing is removed, no flag changes meaning, a 1.x baseline keeps working, and no score changes.

- **Translations** (`translations`), never graded: literal ngx-translate and Transloco keys against the bundles of every language; measure `missing-keys`.
- **Accessibility** (`accessibility`), never graded: unlabelled controls, unnamed buttons, mouse-only `(click)`, images without `alt`, positive `tabindex`; five measures.
- **Test quality** in testing: tests that assert nothing, unwaited assertions in subscriptions, schema-suppressed specs.
- **Subscriptions nothing ends** and **colour literals outside the design tokens** in components.
- **`--since <ref>`:** the findings of one branch.
- **`ng-checkup mcp`:** the MCP server for editor agents.
- **`ng-checkup fleet`:** one page across many repositories.
- **Azure DevOps and Bitbucket** in `init` and `ng add`.
- The Action moves to `@v2` and the GitLab component to `@2`; `@v1` and `@1` stay on 1.6.1.

Since 1.6.1 the package is published as `ng-checkup-cli`.

27. Related packages

Small, dependency-free Angular libraries by the same author, from the same production codebase:

PACKAGE	WHAT
`ng-signal-toast`	Toast notifications on signals: zoneless, OnPush, keyboard and touch accessible, an optional Material theme.
`ng-number-field`	Number and currency inputs for Angular forms: thousand separators, decimal marker, prefix and suffix while typing, a plain number in the form control, separators from the locale.
`ng-phone-input`	A phone number input for Angular forms on libphonenumber-js, with a searchable country menu, as a control and a validator on a plain `matInput`.

These were published as `@stefankuburovic/ng-toast`, `@stefankuburovic/ng-number-input` and `@stefankuburovic/ng-tel-input` before; the code and API are unchanged.

ng-checkup is MIT licensed. Issues and ideas: github.com/stefankuburovic/ng-checkup/issues. A free Angular Health Check: stefankuburovic.github.io.